

3-Day MediaPipe Sprint & Project Memory Kit — CSE498R Sign-Language-to-Voice Project (NSU)

TL;DR

- Use the modern MediaPipe Tasks API (`mp.tasks.vision`), pinned to Python 3.12 in a venv, on `mediapipe==0.10.35` — the legacy `mp.solutions.holistic` API was removed from Python pip wheels starting v0.10.30 and does NOT work in current versions. Do all training on Colab free tier (T4, ~15-30 GPU-hrs/week, allowance unpublished) and run only the live webcam demo locally on the Arch laptop (CPU is fine — MediaPipe was built for it).
- The 3-day plan is achievable: Day 1 = environment + MediaPipe landmark fundamentals + a static-image CNN quick win (Sign Language MNIST); Day 2 = landmark→scikit-learn classifier, then landmark-sequence→LSTM on ~30 self-recorded clips; Day 3 = local live webcam pipeline (hands+face simultaneously) → classifier → on-screen text → **Piper TTS voice output**.
- For voice, recommend **Piper TTS** (fast, offline, CPU-only, MIT-licensed core, has a Bangla `bn_BD` voice) for the final low-latency demo, with `pytttsx3` (**espeak-ng backend**) as the zero-setup fallback. All tools are 100% free.

Key Findings

The single most important technical decision

The biggest trap in this project is the MediaPipe API split. As of mid-2026:

- **Legacy Solutions API** (`mp.solutions.hands`, `mp.solutions.holistic`) — the API used in almost every YouTube sign-language tutorial (including the famous Nicholas Renotte "Action Detection" LSTM tutorial) — **was removed from the Python pip wheels starting version 0.10.30 (December 2025)** and is still gone in the current 0.10.35. This is documented in google-ai-edge/mediapipe GitHub Issue #6192 ("mediapipe 0.10.31 for python does not contain mediapipe.solutions"), where a fresh `pip install mediapipe` (0.10.31) exposes only `mediapipe.Image`, `mediapipe.ImageFormat`, and `mediapipe.tasks` — no `solutions`. Calling `mp.solutions.holistic` on a fresh install raises `AttributeError: module 'mediapipe' has no attribute 'solutions'`. (Qiita)
- **Tasks API** (`mp.tasks.vision.HandLandmarker`, `FaceLandmarker`, `PoseLandmarker`, `HolisticLandmarker`) is the current, supported path. `HolisticLandmarker` was itself removed from Python in v0.10.32 then re-added in v0.10.33, but it has a history of Python runtime bugs, so **the recommended, robust approach is to run** `HandLandmarker` + `FaceLandmarker` (and optionally `PoseLandmarker`) **as separate Tasks in the same loop** — this directly satisfies the "hands AND head/face simultaneously" core requirement and is more stable than Holistic.

Two viable strategies — decide via ADR-0001:

1. **(Recommended) Modern Tasks API path:** `mediapipe==0.10.35`, Python 3.12, run HandLandmarker + FaceLandmarker together. Downside: you must download `.task` model files and adapt tutorial code.
2. **(Fallback for tutorial-copying) Legacy pinned path:** `mediapipe==0.10.21` on Python 3.11/3.12, which still has `mp.solutions.holistic`. This lets you follow Renotte-style tutorials verbatim (543 landmarks in one call), but you are on a frozen old version. Use this ONLY if you get stuck adapting Tasks API code.

Environment facts (verified mid-2026)

- **MediaPipe 0.10.35** (released 27 April 2026) officially supports **Python 3.9–3.12; NOT 3.13** (PyPI classifiers list only 3.9–3.12). Arch ships Python 3.13/3.14 by default, so you **MUST** create a venv with Python 3.12.
- The 0.10.35 wheels are `py3-none` tagged (they may physically install on 3.13), but Google only tested through 3.12 — pin 3.12 for reliability.
- **Colab free tier (2026):** NVIDIA T4 (16 GB VRAM), ~12.7 GB system RAM, up to 12-hour sessions, ~90-min idle timeout, weekly GPU allowance **unpublished and demand-dependent (recent community estimates ~15–30 GPU-hrs; at peak a free session can drop to CPU)**. Disk is ephemeral — mount Google Drive and save checkpoints there.
`AIMultiple`
- **Kaggle Notebooks** as backup: per Kaggle's official docs, "you can use up to 30 hours per week of GPU, and individual sessions can run up to **9 hours**" (not 12h), on P100 (16 GB) or 2×T4; 20 GB persistent storage; predictable quota counter. Better than Colab for long/background runs.
- **MediaPipe live webcam does NOT work in Colab** (`cv2.VideoCapture(0)` can't reach the cloud VM's camera). Colab can only grab still frames or a video stream via a JavaScript `getUserMedia` snippet. Therefore live inference belongs on the Arch laptop.

Datasets (all free)

- **Static image (Task 1):** Sign Language MNIST (Kaggle, `datamunge/sign-language-mnist`) — 27,455 train / 7,172 test, 28×28 grayscale, 24 static ASL letters (no J/Z). Tiny, trains in minutes on T4. Alternative: ASL Alphabet (`grassknotted/asl-alphabet`), 87,000 images, 29 classes).
- **Landmark (Task 2):** Reuse ASL alphabet images → run HandLandmarker → 63-value vectors (21×3). Or self-record.
- **Dynamic sequences (Task 3):** Self-record ~30 clips × 3–5 signs with your webcam (the standard tutorial approach) — most reliable for a 3-day sprint. Public option for Bangla context: BdSLW60 or BdSL40 (see below).
- **Bangla context for the 50 GB dataset:** Likely one of:
 - **BdSL40** — 611 videos over 40 BdSL words, 8–22 clips per word (arXiv:2401.12210; GitHub `Patchwork53/BdSL40_Dataset_AI_for_Bangla_2.0_Honorable_Mention`)).

- **BdSLW60** — per Rubaiyeat et al. (arXiv:2402.08635; *Multimedia Tools and Applications* 84, 42399–42423, 2025): "60 Bangla sign words, with a significant scale of 9307 video trials provided by 18 signers under the supervision of a sign language professional."
- **BdSLW401** — per its authors: "a large-scale, multi-view, word-level Bangla Sign Language dataset with 401 signs and 102,176 video samples from 18 signers in front and lateral views" (arXiv:2503.02360).
- Image-based: **Ishara-Lipi** (1,800 isolated-character images, 36 BdSL characters), **BdSL49**, **BdSL36**.
- If it's ~50 GB it is almost certainly **video-based word-level** (like BdSLW401), which means your main project will need the landmark-sequence + RNN/Transformer approach — exactly what Day 2/Day 3 rehearse.

TTS options (all free)

- **Piper TTS (recommended for final demo):** neural, fully offline, CPU-only, real-time; installed via `(pip install piper-tts)`; has a Bangla voice. Licensing nuance (Rhasspy maintainer, piper GitHub Discussion #271): the Piper project itself is MIT and "can be used in any other projects for any purpose, even commercially... However, most voices use the espeak-ng phonemizer from piper-phonemizer, which has a GPL license." This is fine for a free academic project. Best latency/quality trade-off for CPU. (Tts)
- **pyttsx3 (recommended fallback):** offline wrapper around espeak-ng on Linux; robotic but zero-latency and trivial; needs `(espeak-ng)` installed via pacman. (Pyttsx3)
- **gTTS:** good quality, but needs internet and has latency (a Google request per phrase) — fine for demos, not ideal for live.
- **Browser SpeechSynthesis:** only if you build a web UI.

Details

PART A — THE 3-DAY PLAN

Day 0 (30 min, night before): accounts & downloads

- Create/verify Google account for Colab + Google Drive; create Kaggle account and verify phone (needed to enable GPU) and generate a Kaggle API token (`(kaggle.json)`).
- On Arch: confirm you have a webcam and audio working.

DAY 1 — Environment + MediaPipe fundamentals + static CNN quick win

Block 1 (morning, ~1.5h): Arch Linux venv setup

```
bash
```

```
# 1. Install Python 3.12 + system deps (MediaPipe needs 3.9-3.12; Arch default is
sudo pacman -S --needed python312 base-devel espeak-ng ffmpeg portaudio
# If python312 isn't in the official repo, install from AUR:
# yay -S python312      (or build python3.12 via pyenv)

# 2. Create and activate a venv pinned to 3.12
python3.12 -m venv ~/venvs/slr
source ~/venvs/slr/bin/activate
python --version      # must print Python 3.12.x

# 3. Upgrade pip and install the stack (CPU-only)
pip install --upgrade pip wheel
pip install mediapipe==0.10.35 opencv-python numpy scikit-learn
pip install pyttsx3      # TTS fallback (uses espeak-ng)
pip install piper-tts     # TTS recommended (neural, offline)
pip install tensorflow-cpu # or: pip install torch --index-url https://download

# 4. Sanity checks
python -c "import mediapipe as mp; print(mp.__version__)"      # 0.10.35
python -c "import mediapipe as mp; print(hasattr(mp, 'solutions'))" # False - exp
python -c "import cv2, sklearn, numpy; print('ok')"
espeak-ng "hello world"      # should speak
```

Pitfall: if `pip install mediapipe==0.10.35` fails with "no matching distribution", your venv is not Python 3.12 — recreate it. On Arch, install `espeak-ng` via pacman; pyttsx3 finds it automatically.

Block 2 (late morning, ~2h): Colab notebook + MediaPipe Tasks landmarks Create a Colab notebook `01_mediapipe_fundamentals.ipynb`. Setup cell:

```
python

!pip -q install mediapipe==0.10.35
# Download the Tasks .task model files
!wget -q -O hand_landmarker.task https://storage.googleapis.com/mediapipe-models/h
!wget -q -O face_landmarker.task https://storage.googleapis.com/mediapipe-models/f
!wget -q -O pose_landmarker.task https://storage.googleapis.com/mediapipe-models/p
```

Run HandLandmarker + FaceLandmarker on a still image (IMAGE mode), print and visualize landmarks:

```
python
```

```
import mediapipe as mp, cv2, numpy as np
from mediapipe.tasks import python
from mediapipe.tasks.python import vision

hand = vision.HandLandmarker.create_from_options(vision.HandLandmarkerOptions(
    base_options=python.BaseOptions(model_asset_path='hand_landmarker.task'),
    num_hands=2, running_mode=vision.RunningMode.IMAGE))
face = vision.FaceLandmarker.create_from_options(vision.FaceLandmarkerOptions(
    base_options=python.BaseOptions(model_asset_path='face_landmarker.task'),
    num_faces=1, running_mode=vision.RunningMode.IMAGE))

img = mp.Image.create_from_file('test.jpg') # RGB expected
hres = hand.detect(img); fres = face.detect(img)
print("hands:", len(hres.hand_landmarks), "face pts:", len(fres.face_landmarks[0]))
```

Learn the coordinate format (write in your notes):

- Hand: **21 landmarks** per hand, each (x, y, z) ; x,y normalized to [0,1] by image width/height; z is depth relative to wrist. Wrist=0, thumb=1-4, index=5-8, middle=9-12, ring=13-16, pinky=17-20. [Google](#) [Medium](#)
- Face: **478 landmarks** (FaceLandmarker; 468 in older FaceMesh) (x, y, z) normalized.
- Pose: **33 landmarks**, normalized $(x, y, z, visibility)$.
- MediaPipe Holistic (legacy) reported 543 total (33 pose + 468 face + 21×2 hands). [GitHub](#)

Block 3 (afternoon, ~2.5h): static-image CNN quick win on T4 In Colab, set Runtime → Change runtime type → T4 GPU. Notebook [02_static_cnn.ipynb](#):

```
python

# Get Sign Language MNIST via Kaggle API
from google.colab import files; files.upload() # upload kaggle.json
!mkdir -p ~/.kaggle && cp kaggle.json ~/.kaggle/ && chmod 600 ~/.kaggle/kaggle.json
!kaggle datasets download -d datamunge/sign-language-mnist && unzip -q sign-langua
```

Train a tiny CNN (2 conv blocks) in Keras — this dataset reaches >95% test accuracy in a few epochs on T4 in under 5 minutes. Save [models/static_cnn.keras](#) to Drive. **Goal of the day:** you've seen (a) landmarks and (b) a raw-pixel classifier, and understand why landmarks are more robust to background/lighting.

Common Day-1 pitfalls: BGR vs RGB (OpenCV loads BGR; MediaPipe wants RGB — convert with [cv2.cvtColor\(frame, cv2.COLOR_BGR2RGB\)](#) then wrap as [mp.Image\(image_format=mp.ImageFormat.SRGB, data=rgb\)](#)); forgetting to set GPU runtime; Drive not mounted so checkpoints vanish.

DAY 2 — Landmark→classifier, then landmark-sequences→LSTM

Block 1 (morning, ~2.5h): landmark-based static classifier (scikit-learn) Notebook

`03_landmark_classifier.ipynb`. For each ASL-alphabet image: run HandLandmarker → flatten $21 \times 3 = 63$ -value feature vector → build X ($n_{\text{samples}} \times 63$), y (labels). Train `RandomForestClassifier` or a small `MLPClassifier`; evaluate with a confusion matrix. This trains in seconds on CPU (no GPU needed) and typically beats the raw-pixel CNN on robustness. Normalize landmarks (subtract wrist coordinate, scale by hand size) so the classifier is translation/scale invariant. Save `models/landmark_rf.joblib`.

Block 2 (afternoon, ~3.5h): dynamic sign sequences → LSTM Notebook

`04_dynamic_lstm.ipynb`. Follow the Renotte "Action Detection" methodology but with the Tasks API:

- Collect:** Choose 3–5 dynamic signs (e.g., "hello", "thanks", "iloveyou"). Record **30 sequences per sign, 30 frames per sequence**. Per frame, extract a fixed-length keypoint vector: hands ($21 \times 3 \times 2 = 126$) + a face subset or pose (add ~132 pose or a face slice). Save as `.numpy` arrays: shape `(num_sequences, 30, features)`.
 - Do collection on the **Arch laptop** (needs webcam) with a local script; upload the `.numpy` folder to Drive. OR record short videos and extract landmarks in Colab in VIDEO mode.
- Train LSTM in Colab (T4):** small stacked LSTM (e.g., $64 \rightarrow 128 \rightarrow 64$ units) + Dense softmax; `categorical_crossentropy`, Adam; ~200 epochs on this tiny set takes minutes. Tips from a public reproduction of the Renotte approach: **tanh activation works better than relu with LSTMs**, and **dropping noisy face landmarks can help** if the face adds only noise — but keep face for your project since head/face tracking is a core requirement (facial grammar matters in sign language). Save `models/lstm_signs.keras`. [GitHub](#)

VIDEO-mode pitfall (critical): In VIDEO or LIVE_STREAM running mode, every `detect_for_video(img, timestamp_ms)` call needs a **strictly monotonically increasing timestamp in milliseconds**. If timestamps don't increase you get a `ValueError`. Derive it from a frame counter or `time.time()*1000`. [Google AI](#)

DAY 3 — Local live demo (hands+face) → text → voice

Block 1 (morning, ~1h): Piper TTS setup on Arch

```
bash

source ~/venvs/slr/bin/activate
# English voice
wget -O en_US-amy-medium.onnx "https://huggingface.co/rhasspy/piper-voices/resolve
wget -O en_US-amy-medium.onnx.json "https://huggingface.co/rhasspy/piper-voices/re
echo "hello, sign language recognized" | piper -m en_US-amy-medium.onnx --output_f
```

For Bangla output, download the `bn` voice from the Piper voices repo (browse `rhasspy/piper-voices` on Hugging Face for the `bn` folder).

Block 2 (midday, ~3h): wire the end-to-end live pipeline Local script `src/live_demo.py`. Loop:

```
webcam frame (OpenCV, BGR) → convert RGB → mp.Image
→ HandLandmarker.detect_for_video(img, ts)    # 21×3 ×2 hands
→ FaceLandmarker.detect_for_video(img, ts)    # 478×3 (or subset)
→ assemble feature vector (same layout as training)
→ [static] landmark_rf.predict() OR [dynamic] buffer last 30 frames →
lstm.predict()
→ if confidence > threshold and label changed → show text on frame
(cv2.putText)
    AND speak via Piper (run TTS in a background thread so it doesn't block
the video loop)
```

Key implementation notes:

- Use **VIDEO** running mode with monotonic ms timestamps (simpler than LIVE_STREAM async callbacks for a first demo).
- **Debounce speech:** only speak when the predicted label is stable for N frames and differs from the last spoken label, else it repeats every frame.
- **Threaded TTS:** call Piper/pyttsx3 in a `threading.Thread` or queue so synthesis doesn't freeze the webcam loop.
- Expected performance: on the Ryzen 5 5500U CPU, HandLandmarker+FaceLandmarker together should run at usable interactive frame rates (MediaPipe is CPU-optimized); if slow, drop face to a landmark subset or process every other frame.

Block 3 (afternoon, ~2h): document learnings & update memory files Fill in

`agent_docs/milestone_log.md`, `changelog.md`, `test_log.md`, and add any new ADRs. Record measured FPS, accuracy, and what broke.

Hardware note: The Windows desktop (RX 570, no CUDA) can run everything CPU-side identically but cannot use NVIDIA GPU acceleration for TensorFlow/PyTorch — do training on Colab/Kaggle regardless. The RX 570 does not help TF/PyTorch without ROCm (not worth configuring for this sprint).

PART B — PROJECT MEMORY / DOCUMENTATION FILES (copy verbatim into repo)

Suggested repo layout

```
slr-voice/
├── CLAUDE.md
├── constitution.md
├── README.md
├── notebooks/
│   ├── 01_mediapipe_fundamentals.ipynb
│   ├── 02_static_cnn.ipynb
│   ├── 03_landmark_classifier.ipynb
│   └── 04_dynamic_lstm.ipynb
├── src/
│   ├── extract_landmarks.py
│   ├── collect_sequences.py
│   ├── train_landmark_clf.py
│   ├── live_demo.py
│   └── tts.py
├── data/          # gitignored; raw + processed (npz)
├── models/        # .keras / .joblib / .task
└── agent_docs/
    ├── milestone_log.md
    ├── current_task.md
    ├── changelog.md
    ├── test_log.md
    ├── decisions.md
    ├── codebase_map.md
    └── session_protocol.md
```

CLAUDE.md

markdown

CLAUDE.md – Project Guide for Claude Code

Project

Sign-Language-to-Voice recognition for CSE498R (North South University).

Pipeline: Raw media (webcam/video) → MediaPipe landmark processing → classifier → Core differentiator: tracks BOTH hands AND head/face simultaneously (not hands-onl

Current Phase

3-day learning sprint (mini-project) before the main capstone. Goal: experience al major approaches – static image CNN, landmark classifier, dynamic LSTM, live webca

Stack (100% free)

- Python 3.12 (venv) – MediaPipe requires 3.9-3.12, NOT 3.13.
- MediaPipe 0.10.35 **Tasks API** (`mp.tasks.vision`) – HandLandmarker + FaceLandm
- OpenCV, NumPy, scikit-learn, TensorFlow-CPU (or PyTorch-CPU).
- TTS: Piper (primary), pyttsx3/espeak-ng (fallback).
- Training on Google Colab (T4) / Kaggle (P100); live inference LOCAL on Arch lapt

Hard Constraints

- Free tools only. CPU-friendly locally (no CUDA – Arch laptop has no GPU; desktop
- Legacy `mp.solutions.*` is REMOVED from pip >=0.10.30 – DO NOT use it. Use Tasks
- Live webcam does not work in Colab – keep live code local.

Workflow Rules (STRICT – plan-first)

1. **Plan before code.** For any task, first write a short plan (files to touch, a risks) and WAIT for my explicit "approved" before writing code.
2. **Small, reviewable changes.** One concern per change; show diffs; explain why.
3. **No silent scope creep.** If you discover extra work, propose it as a new task
4. **Cite decisions.** Any architectural choice must reference or create an ADR in
5. **Update memory.** At end of every session, update changelog.md, current_task.m milestone_log.md (see session_protocol.md).
6. **Ask when unsure.** If a fact is uncertain (API, dataset type), say so; don't

Read Order at Session Start

CLAUDE.md → constitution.md → agent_docs/current_task.md → agent_docs/decisions.md

constitution.md

markdown

constitution.md – Non-Negotiable Principles

These principles override convenience. Violating one requires an ADR that explicit supersedes this file.

1. ****Free only.**** Every tool, dataset, model, and service must be free (no paid API, no trial credit cards). Colab/Kaggle free tiers are the compute budget.
2. ****CPU-friendly local.**** All code that runs on my machines must run on CPU. The laptop (Ryzen 5 5500U, 12 GB RAM, integrated GPU) and Windows desktop (Ryzen 5 24 GB, AMD RX 570) have NO usable CUDA. GPU work happens only on Colab/Kaggle.
3. ****Colab-first training.**** Notebooks and training run on Colab/Kaggle; checkpoint to Google Drive. Local machine is for live webcam inference and development only.
4. ****Hands + head/face simultaneously is CORE.**** The system MUST track both hands face/head at the same time. Any design that drops face tracking violates the purpose (facial expression is grammatically meaningful in sign language).
5. ****Voice output is CORE.**** Final output MUST be spoken audio (TTS), not only on-
6. ****Modern MediaPipe Tasks API.**** Use ``mp.tasks.vision.*``. The legacy ``mp.solutions`` is removed from current pip wheels and is forbidden except behind a pinned-version.
7. ****Reproducible.**** Pin versions (mediapipe==0.10.35, Python 3.12). Record dataset and preprocessing so results reproduce.
8. ****Plan before code; approve before change.**** No code is written without an approved plan.

agent_docs/current_task.md (pre-filled with Day 1)

markdown

Current Task

Active: Day 1 – Environment + MediaPipe fundamentals + static CNN

Definition of Done

- [] Arch venv (Python 3.12) created; mediapipe 0.10.35 imports; ``hasattr(mp,'sol`
- [] espeak-ng speaks from terminal; pyttsx3 speaks from Python.
- [] Colab notebook 01 runs HandLandmarker + FaceLandmarker on a test image; land
- [] Coordinate format documented in notes (normalized x,y; z depth; landmark ind
- [] Colab notebook 02 trains a tiny CNN on Sign Language MNIST to >90% test acc

Plan (approve before coding)

1. Write setup commands into README (venv + pip). [done in guide]
2. Notebook 01: model download cell + IMAGE-mode detection + visualization.
3. Notebook 02: Kaggle download + CNN + save.

Notes / Blockers

- Remember GPU runtime in Colab (Runtime → Change runtime type → T4).
- Mount Drive first; ephemeral disk wipes on disconnect.

Next: Day 2 – landmark→sklearn classifier, then landmark-sequences→LSTM

Then: Day 3 – live local webcam (hands+face) → text → Piper TTS

agent_docs/decisions.md (ADR-style, pre-seeded)

markdown

Architecture Decision Records

ADR-0001: Use MediaPipe Tasks API (HandLandmarker + FaceLandmarker), not legacy

- Status: Accepted
- Context: The legacy `mp.solutions.holistic/hands` API used in most tutorials was from MediaPipe Python pip wheels starting v0.10.30 (Dec 2025); it errors on 0.10 (see GitHub issue #6192). HolisticLandmarker (Tasks) was removed in 0.10.32 and in 0.10.33 but has a history of Python runtime bugs.
- Decision: Use the Tasks API and run HandLandmarker + FaceLandmarker (+PoseLandma needed) as separate tasks in one loop. This is stable and satisfies hands+face-s
- Consequences: Must download .task model files; must adapt tutorial code from Sol
- Fallback: If blocked, pin mediapipe==0.10.21 (Python 3.11/3.12) to use legacy Ho

ADR-0002: Colab/Kaggle for training; local Arch laptop for live inference

- Status: Accepted
- Context: Local machines have no usable CUDA. Colab free gives a T4; Kaggle gives 30h/week (9h max per session). MediaPipe live webcam cannot access the camera in
- Decision: All training + heavy notebooks on Colab (T4) with Kaggle as backup; ch Google Drive. Live webcam demo runs locally on the Arch laptop (CPU).
- Consequences: Need a clean model-handoff (download .keras/.joblib from Drive to

ADR-0003: Piper TTS as primary voice engine; pyttsx3 fallback

- Status: Accepted
- Context: Need free, low-latency, offline voice on CPU. Piper is neural, offline, real-time, MIT-licensed core (voices use GPL espeak-ng phonemizer – fine for aca and has a Bangla voice. pyttsx3 (espeak-ng) is trivial but robotic. gTTS needs i
- Decision: Use Piper for the demo; keep pyttsx3 as a zero-dependency fallback. Ru background thread to avoid blocking the webcam loop.
- Consequences: Must download Piper .onnx voice files; add threaded speech + debu

ADR-0004: Landmark-based (not raw-pixel) representation for dynamic signs

- Status: Accepted
- Context: Raw-pixel video models (3D-CNN) are heavy, background/lighting sensitiv on CPU. Landmark sequences are compact (~hundreds of floats/frame), robust, and
- Decision: For dynamic signs, extract MediaPipe landmark sequences (hands+face/po an LSTM/GRU on the sequences. Raw-pixel CNN is used only for the Day-1 static qu
- Consequences: Depends on MediaPipe detection quality; must handle missing-hand f (zero-fill or interpolate).

agent_docs/milestone_log.md

markdown

Milestone Log

(append-only; newest at bottom. Format: DATE - milestone - evidence)

- 2026-__-__ - Repo + memory files created - CLAUDE.md, constitution.md, agent_doc
- (Day 1) - Environment verified; landmarks visualized; static CNN >90% - notebook
- (Day 2) - Landmark classifier + dynamic LSTM trained - notebooks 03,04; models s
- (Day 3) - Live hands+face→text→voice demo working - src/live_demo.py; recorded c

agent_docs/changelog.md

markdown

Changelog

(newest at top. One entry per session. Format below.)

[Unreleased]

Added

- Initial repo scaffold, memory files, and 3-day plan.

Changed

Fixed

Decisions

- ADR-0001..0004 seeded.

agent_docs/test_log.md

markdown

Test Log

(What was tested, how, result. Manual is fine for this project.)

	Date
	Component
	Test
	Result
	Notes

	venv
	`import mediapipe`
	on Py3.12
	pending
	expect 0.10.35
	landmarks
	detect on test.jpg
	pending
	expect 21/hand, 478 face

|
|
static CNN
|
test accuracy
|
pending
|
target >90%
|
|
|
landmark clf
|
confusion matrix
|
pending
|
|
|
|
|
LSTM
|
val accuracy on held-out sequences
|
pending
|
|
|
|
live demo
|
FPS + correct speech
|
pending
|
measure on Ryzen 5 5500U
|

agent_docs/codebase_map.md

markdown

Codebase Map

- ``notebooks/01_mediapipe_fundamentals.ipynb`` – download .task models; IMAGE-mode
- ``notebooks/02_static_cnn.ipynb`` – Sign Language MNIST; tiny CNN; saves models/st
- ``notebooks/03_landmark_classifier.ipynb`` – images→landmarks (63-vec)→RandomFores
- ``notebooks/04_dynamic_lstm.ipynb`` – landmark sequences (30 frames)→LSTM; saves m
- ``src/extract_landmarks.py`` – reusable Hand+Face landmark extraction → feature ve
- ``src/collect_sequences.py`` – webcam recorder: 30 seqs × 30 frames per sign → dat
- ``src/train_landmark_clf.py`` – local training of sklearn classifier (CPU).
- ``src/live_demo.py`` – webcam → Hand+Face → classifier/LSTM → on-screen text → thr
- ``src/tts.py`` – Piper wrapper + pyttsx3 fallback; debounced speak().
- ``models/`` – trained artifacts + .task files. ``data/`` – datasets (gitignored).

agent_docs/session_protocol.md

markdown

Session Protocol

Start of EVERY Claude Code session (paste this):

"Read, in order: CLAUDE.md, constitution.md, agent_docs/current_task.md, agent_docs/decisions.md, agent_docs/codebase_map.md. Summarize the active task and constraints back to me in 5 bullets. Then propose a PLAN for today's task and WAIT approval before writing any code."

During session

- Keep changes small; show diffs; one concern at a time.
- If a decision is architectural, write/update an ADR in decisions.md.
- If blocked or a fact is uncertain, stop and ask.

End of EVERY session (the ritual)

1. Update ``changelog.md`` (Added/Changed/Fixed/Decisions for today).
2. Update ``current_task.md`` (check off Done items; set the next Active task + plan)
3. If a milestone was hit, append to ``milestone_log.md`` with evidence (file/commit)
4. Add rows to ``test_log.md`` for anything tested.
5. Commit: ``git add -A && git commit -m "session: <summary>"``.
6. Print a 3-line summary: what changed, what's next, any blockers.

PART C — HOW TO USE THESE FILES WITH CLAUDE CODE

One-time setup (first session)

1. Create the repo and files:


```
bash

mkdir slr-voice && cd slr-voice && git init
mkdir -p notebooks src data models agent_docs
# create the files above (paste contents), then:
printf "data/\nmodels/*.task\n__pycache__/\n*.pyc\n.venv/\n" > .gitignore
git add -A && git commit -m "scaffold: memory files + 3-day plan"
```

2. Open Claude Code in the `slr-voice/` folder.
3. Paste this as your FIRST message:

"This project uses a plan-first, approve-before-code workflow with ADR-style decisions. Read these files in order and confirm you understand: CLAUDE.md, constitution.md, agent_docs/current_task.md, agent_docs/decisions.md, agent_docs/codebase_map.md, agent_docs/session_protocol.md. Then summarize back to me: (a) the project goal and pipeline, (b) the 6 hard constraints, (c) today's active task, (d) the read-order and end-of-session ritual. Do NOT write any code yet — after your summary, propose a plan for Day 1 Block 2 (Notebook 01) and wait for my explicit 'approved!'"

Every subsequent session

- Paste the "Start of EVERY session" block from `session_protocol.md`.
- Review Claude's plan, reply `approved` (or request changes) before it writes code.
- Work in small diffs.
- Before you stop, tell Claude: "Run the end-of-session ritual from session_protocol.md" — it will update changelog/current_task/milestone_log/test_log and commit.

Why this works with your workflow

It mirrors your other capstone's plan-first/ADR discipline: the constitution encodes non-negotiables so Claude can't quietly break them; current_task.md keeps a single active focus; decisions.md gives an auditable trail; the session protocol makes start/end deterministic so context survives across Claude Code sessions.

Recommendations

1. **Start today with the Arch venv on Python 3.12 and confirm `mediapipe==0.10.35` imports and `mp.solutions` is absent** — this single check prevents days of following dead tutorials. If you cannot get Python 3.12 on Arch quickly, use pyenv.
2. **Adopt the Tasks-API path (ADR-0001) by default.** Only fall back to `mediapipe==0.10.21` legacy Holistic if adapting tutorial code stalls you for more than ~2 hours.
3. **Do all training on Colab; keep live webcam local.** Save every model to Google Drive immediately (Colab disk is ephemeral). If Colab denies GPU at peak, switch to Kaggle (predictable 30h/week, 9h max per session).

4. **For dynamic signs, self-record 30×30-frame clips of 3–5 signs** rather than wrangling a big dataset — it's the fastest route to a working LSTM in the sprint window.
5. **Use Piper for the final voice demo** with threaded, debounced speech; keep pyttsx3 wired as fallback.
6. **When the real 50 GB dataset arrives, classify it first:** if video/word-level (likely, given size — cf. BdSLW401's 102,176 videos), your Day-2/Day-3 landmark-sequence pipeline is the template; if image/alphabet (cf. Ishara-Lipi), the Day-1/Day-2 static pipelines apply. Update ADRs accordingly.

Thresholds that change the plan: if local live FPS is unusably low ($< \sim 8$ FPS) with Hand+Face, drop face to a small landmark subset or process every 2nd frame; if landmark classifier accuracy is poor, add wrist-relative normalization; if Colab GPU is unavailable repeatedly, move to Kaggle.

Caveats

- MediaPipe versioning is volatile: `mp.solutions` removal (0.10.30) and HolisticLandmarker's removal (0.10.32)/re-add (0.10.33) happened within months. Re-verify `mp.__version__` behavior on install day; pin versions. (Note: 0.10.34 does not appear on PyPI — the history jumps 0.10.33 $\rightarrow$ 0.10.35.)
- The 0.10.35 wheel is `py3-none` tagged and may physically install on Python 3.13/3.14, but Google only tested through 3.12 — 3.13 is unsupported and may fail at runtime. Pin 3.12.
- Colab free-tier limits are unpublished and dynamic (~ 15 – 30 GPU-hrs/week is a community estimate, not an official number); GPU is not guaranteed at peak.
- Piper Bangla voice quality and exact availability should be verified on install day from the piper-voices repo; if the Bangla voice is weak, output romanized/English text to TTS or use gTTS Bangla (needs internet). Piper voices use the GPL-licensed espeak-ng phonemizer even though Piper's core is MIT — no problem for a free academic project, but note it if you ever redistribute.
- The 50 GB dataset type is unknown; this plan deliberately covers both image and video paths so you are not blocked either way.
- HolisticLandmarker (Tasks, Python) re-add in 0.10.33 fixed the module availability but the release notes did not confirm all earlier Python runtime bugs are resolved — another reason to prefer separate Hand+Face landmarkers.
- The Renotte-style "Action Detection" tutorial and most GitHub reproductions use the now-removed `mp.solutions.holistic` API — treat their MediaPipe calls as reference only and translate them to the Tasks API.